

```
import os

PROJECT_DIR = "/content/ai_advisory_blockchain"
os.makedirs(PROJECT_DIR, exist_ok=True)

print("Project folder created:", PROJECT_DIR)
```

Project folder created: /content/ai_advisory_blockchain

```
import os

PROJECT_DIR = "/content/ai_advisory_blockchain"

# -----
# stock_universe.py
# -----
stock_universe_code = '''
STOCK_UNIVERSE = {
    "PAYFIN": {"beta": 1.35, "analyst_expected_return": 0.16, "std_dev": 0.28},
    "PAYRETAIL": {"beta": 0.85, "analyst_expected_return": 0.11, "std_dev": 0.17},
    "PAYINFRA": {"beta": 1.10, "analyst_expected_return": 0.135, "std_dev": 0.22},
    "PAYGOLD": {"beta": 0.20, "analyst_expected_return": 0.08, "std_dev": 0.12},
    "PAYBOND": {"beta": 0.05, "analyst_expected_return": 0.065, "std_dev": 0.04},
    "PAYTECH": {"beta": 1.55, "analyst_expected_return": 0.19, "std_dev": 0.34},
}

RISK_FREE_RATE = 0.07
MARKET_RETURN = 0.13
'''

with open(os.path.join(PROJECT_DIR, "stock_universe.py"), "w") as f:
    f.write(stock_universe_code.strip())

# -----
# investor_profiles.py
# -----
investor_profiles_code = '''
INVESTOR_PROFILES = [
    {"investor_id": "INV01", "risk_tolerance": "Conservative", "horizon_years": 3, "investment_strategy": "Conservative"},
    {"investor_id": "INV02", "risk_tolerance": "Moderate", "horizon_years": 7, "investment_strategy": "Moderate"},
    {"investor_id": "INV03", "risk_tolerance": "Aggressive", "horizon_years": 12, "investment_strategy": "Aggressive"},
    {"investor_id": "INV04", "risk_tolerance": "Moderate", "horizon_years": 5, "investment_strategy": "Moderate"},
    {"investor_id": "INV05", "risk_tolerance": "Aggressive", "horizon_years": 2, "investment_strategy": "Aggressive"},
]
'''

with open(os.path.join(PROJECT_DIR, "investor_profiles.py"), "w") as f:
    f.write(investor_profiles_code.strip())

# -----
# disclosure_snippets.py
# -----
disclosure_snippets_code = '''
DISCLOSURE_SNIPPETS = [
    "doc_01: Assuming input costs remain stable through the next two quarters, we expect margins to improve.",
    "doc_02: The company faces an ongoing litigation matter related to a former vendor contract, which may impact future earnings.",
    "doc_03: Our top three customers together account for approximately 42 percent of total revenue.",
    "doc_04: We remain cautiously optimistic about demand recovery, though visibility beyond the next quarter is limited.",
    "doc_05: The board is confident in the long-term strategy and has approved an expanded share repurchase program.",
]
'''
```

```

        "doc_06: A recent regulatory notice has been received regarding data-localization comp[
    ]
    ...

with open(os.path.join(PROJECT_DIR, "disclosure_snippets.py"), "w") as f:
    f.write(disclosure_snippets_code.strip())

print("✅ Three seed files created successfully.")

```

✅ Three seed files created successfully.

```

import sys

sys.path.insert(0, PROJECT_DIR)

from stock_universe import STOCK_UNIVERSE, RISK_FREE_RATE, MARKET_RETURN
from investor_profiles import INVESTOR_PROFILES
from disclosure_snippets import DISCLOSURE_SNIPPETS

print("=== SEED DATA VERIFICATION ===")
print("Stocks:", len(STOCK_UNIVERSE))
print("Investors:", len(INVESTOR_PROFILES))
print("Disclosure snippets:", len(DISCLOSURE_SNIPPETS))

print("\nStock tickers:")
print(list(STOCK_UNIVERSE.keys()))

print("\nInvestor IDs:")
print([x["investor_id"] for x in INVESTOR_PROFILES])

print("\nDisclosure IDs:")
print([x.split(":")[0] for x in DISCLOSURE_SNIPPETS])

```

```

=== SEED DATA VERIFICATION ===
Stocks: 6
Investors: 5
Disclosure snippets: 6

Stock tickers:
['PAYFIN', 'PAYRETAIL', 'PAYINFRA', 'PAYGOLD', 'PAYBOND', 'PAYTECH']

Investor IDs:
['INV01', 'INV02', 'INV03', 'INV04', 'INV05']

Disclosure IDs:
['doc_01', 'doc_02', 'doc_03', 'doc_04', 'doc_05', 'doc_06']

```

```

advisory_agent_code = ''
import os
import sys

PROJECT_DIR = "/content/ai_advisory_blockchain"

if PROJECT_DIR not in sys.path:
    sys.path.insert(0, PROJECT_DIR)

from stock_universe import STOCK_UNIVERSE, RISK_FREE_RATE, MARKET_RETURN
from investor_profiles import INVESTOR_PROFILES

# =====
# PRESCRIBED ALLOCATION TABLE
# =====

```

```

ALLOCATION_TABLE = {
    "Conservative": ["PAYBOND", "PAYGOLD", "PAYRETAIL"],
    "Moderate": ["PAYRETAIL", "PAYINFRA", "PAYGOLD"],
    "Aggressive": ["PAYTECH", "PAYFIN", "PAYINFRA"],
}

# =====
# ACT STAGE - TOOL CALL
# =====

def get_stock_data(ticker):
    """
    Simulated external API/tool call.

    Returns the local stock data for the requested ticker.
    """

    if ticker not in STOCK_UNIVERSE:
        raise ValueError(f"Unknown ticker: {ticker}")

    return {
        "ticker": ticker,
        "beta": STOCK_UNIVERSE[ticker]["beta"],
        "analyst_expected_return": STOCK_UNIVERSE[ticker]["analyst_expected_return"],
        "std_dev": STOCK_UNIVERSE[ticker]["std_dev"],
    }

# =====
# CAPM CALCULATION
# =====

def calculate_capm_return(beta):
    """
    CAPM expected return:

    
$$E(R_i) = R_f + \beta * (R_m - R_f)$$

    """

    return RISK_FREE_RATE + beta * (MARKET_RETURN - RISK_FREE_RATE)

# =====
# PORTFOLIO VARIANCE
# =====

def calculate_portfolio_variance(stock_data, weights, correlation=0.3):
    """
    Portfolio variance:

    
$$\text{Var}(R_p) = \sum w_i^2 \sigma_i^2 + 2 \sum w_i w_j \text{Cov}(R_i, R_j)$$


    where:

    
$$\text{Cov}(R_i, R_j) = \rho * \sigma_i * \sigma_j$$

    """

    variance = 0.0

```

```

# Individual variance terms
for i in range(len(stock_data)):
    sigma_i = stock_data[i]["std_dev"]
    w_i = weights[i]

    variance += (w_i ** 2) * (sigma_i ** 2)

# Pairwise covariance terms
for i in range(len(stock_data)):
    for j in range(i + 1, len(stock_data)):
        sigma_i = stock_data[i]["std_dev"]
        sigma_j = stock_data[j]["std_dev"]

        w_i = weights[i]
        w_j = weights[j]

        covariance = correlation * sigma_i * sigma_j

        variance += 2 * w_i * w_j * covariance

return variance

# =====
# THINK → ACT → OBSERVE
# =====

def advise_investor(investor):
    """
    Runs the complete Think → Act → Observe advisory loop.
    """

    investor_id = investor["investor_id"]
    risk_tolerance = investor["risk_tolerance"]

    # -----
    # THINK
    # -----

    tickers = ALLOCATION_TABLE[risk_tolerance]
    weights = [1 / 3] * 3

    # -----
    # ACT
    # -----

    stock_data = []

    for ticker in tickers:
        stock_data.append(get_stock_data(ticker))

    # -----
    # OBSERVE
    # -----

    capm_returns = []

    for stock in stock_data:
        capm_return = calculate_capm_return(stock["beta"])
        capm_returns.append(capm_return)

    # Equal-weighted expected portfolio return

```

```

portfolio_expected_return = sum(
    weights[i] * capm_returns[i]
    for i in range(3)
)

# Portfolio variance
portfolio_variance = calculate_portfolio_variance(
    stock_data,
    weights,
    correlation=0.3
)

# Portfolio standard deviation
portfolio_std_dev = portfolio_variance ** 0.5

# -----
# HUMAN-IN-THE-LOOP ESCALATION
# -----

escalation_threshold = 0.20

if portfolio_std_dev > escalation_threshold:
    escalation_flag = "ESCALATED_TO_HUMAN_ADVISOR"
else:
    escalation_flag = "FINALIZED"

# -----
# MOCK LLM NARRATIVE
# -----

recommendation = (
    f"For {risk_tolerance} investor {investor_id}, "
    f"we recommend an allocation across {'', '.join(tickers)} "
    f"with an expected portfolio return of "
    f"{portfolio_expected_return:.1%} and volatility of "
    f"{portfolio_std_dev:.1%}."
)

return {
    "investor_id": investor_id,
    "risk_tolerance": risk_tolerance,
    "tickers": tickers,
    "weights": weights,
    "capm_returns": capm_returns,
    "portfolio_expected_return": portfolio_expected_return,
    "portfolio_variance": portfolio_variance,
    "portfolio_std_dev": portfolio_std_dev,
    "escalation": escalation_flag,
    "recommendation": recommendation,
}

# =====
# RUN ALL INVESTORS
# =====

if __name__ == "__main__":

    print("=" * 70)
    print("AI-AUGMENTED PORTFOLIO ADVISORY AGENT")
    print("Think → Act → Observe")
    print("=" * 70)

```

```

results = []

for investor in INVESTOR_PROFILES:

    result = advise_investor(investor)
    results.append(result)

    print(f"\nInvestor: {result['investor_id']}")
    print(f"Risk tolerance: {result['risk_tolerance']}")
    print(f"Allocation: {result['tickers']}")
    print(f"CAPM returns: {[round(x, 4) for x in result['capm_returns']]}")
    print(
        f"Expected portfolio return: "
        f"{result['portfolio_expected_return']:.2%}"
    )
    print(
        f"Portfolio variance: "
        f"{result['portfolio_variance']:.6f}"
    )
    print(
        f"Portfolio standard deviation: "
        f"{result['portfolio_std_dev']:.2%}"
    )
    print(f"Decision: {result['escalation']}")
    print(f"Recommendation: {result['recommendation']}")

    print("\n" + "=" * 70)
    print("ADVISORY RUN COMPLETE")
    print("=" * 70)
    ...

with open("/content/ai_advisory_blockchain/advisory_agent.py", "w") as f:
    f.write(advisory_agent_code.strip())

print("✅ advisory_agent.py created successfully.")

```

✅ advisory_agent.py created successfully.

```

import sys
import os

PROJECT_DIR = "/content/ai_advisory_blockchain"

if PROJECT_DIR not in sys.path:
    sys.path.insert(0, PROJECT_DIR)

from advisory_agent import advise_investor
from investor_profiles import INVESTOR_PROFILES

for investor in INVESTOR_PROFILES:
    result = advise_investor(investor)

    print("=" * 60)
    print("Investor:", result["investor_id"])
    print("Risk:", result["risk_tolerance"])
    print("Allocation:", result["tickers"])
    print("Expected return:", f"{result['portfolio_expected_return']:.2%}")
    print("Portfolio volatility:", f"{result['portfolio_std_dev']:.2%}")
    print("Decision:", result["escalation"])
    print("Recommendation:", result["recommendation"])

```

```

=====
Investor: INV01

```

```

Risk: Conservative
Allocation: ['PAYBOND', 'PAYGOLD', 'PAYRETAIL']
Expected return: 9.20%
Portfolio volatility: 8.44%
Decision: FINALIZED
Recommendation: For Conservative investor INV01, we recommend an allocation across PAYBOND,
=====
Investor: INV02
Risk: Moderate
Allocation: ['PAYRETAIL', 'PAYINFRA', 'PAYGOLD']
Expected return: 11.30%
Portfolio volatility: 12.57%
Decision: FINALIZED
Recommendation: For Moderate investor INV02, we recommend an allocation across PAYRETAIL, PA
=====
Investor: INV03
Risk: Aggressive
Allocation: ['PAYTECH', 'PAYFIN', 'PAYINFRA']
Expected return: 15.00%
Portfolio volatility: 20.58%
Decision: ESCALATED_TO_HUMAN_ADVISOR
Recommendation: For Aggressive investor INV03, we recommend an allocation across PAYTECH, PA
=====
Investor: INV04
Risk: Moderate
Allocation: ['PAYRETAIL', 'PAYINFRA', 'PAYGOLD']
Expected return: 11.30%
Portfolio volatility: 12.57%
Decision: FINALIZED
Recommendation: For Moderate investor INV04, we recommend an allocation across PAYRETAIL, PA
=====
Investor: INV05
Risk: Aggressive
Allocation: ['PAYTECH', 'PAYFIN', 'PAYINFRA']
Expected return: 15.00%
Portfolio volatility: 20.58%
Decision: ESCALATED_TO_HUMAN_ADVISOR
Recommendation: For Aggressive investor INV05, we recommend an allocation across PAYTECH, PA

```

```

# =====
# PART B – STRUCTURED DISCLOSURE EXTRACTION
# =====

import re

def extract_signals(snippet: str) -> dict:
    """
    Extract risk flags, hedging signals, and sentiment
    from a disclosure snippet using deterministic mock rules.
    """

    text = snippet.lower()

    # -----
    # Risk flag detection
    # -----
    risk_flags = []

    if "litigation" in text:
        risk_flags.append("litigation")

    if "regulatory" in text:
        risk_flags.append("regulatory")

    if "customer concentration" in text:
        risk_flags.append("customer concentration")

```

```

# -----
# Hedging detection
# -----
hedging_phrases = [
    "assuming",
    "cautiously",
    "visibility"
]

hedging_detected = any(
    phrase in text for phrase in hedging_phrases
)

# -----
# Sentiment detection
# -----
if "confident" in text or "approved" in text:
    sentiment = "confident"

elif hedging_detected:
    sentiment = "cautious"

else:
    sentiment = "neutral"

return {
    "risk_flags": risk_flags,
    "hedging_detected": hedging_detected,
    "sentiment": sentiment
}

# =====
# RUN AGAINST ALL 6 DISCLOSURE SNIPPETS
# =====

print("=== STRUCTURED DISCLOSURE EXTRACTION ===")

disclosure_results = []

for snippet in DISCLOSURE_SNIPPETS:

    doc_id = snippet.split(":")[0]

    result = extract_signals(snippet)

    disclosure_results.append({
        "doc_id": doc_id,
        "risk_flags": result["risk_flags"],
        "hedging_detected": result["hedging_detected"],
        "sentiment": result["sentiment"]
    })

print("\n" + "=" * 60)
print("Document:", doc_id)
print("Risk flags:", result["risk_flags"])
print("Hedging detected:", result["hedging_detected"])
print("Sentiment:", result["sentiment"])

```

```

=== STRUCTURED DISCLOSURE EXTRACTION ===

```

```

=====
Document: doc_01

```



```
Risk flags: []
Hedging detected: True
Sentiment: cautious
```

```
=====
Document: doc_02
Risk flags: ['litigation']
Hedging detected: False
Sentiment: neutral
```

```
=====
Document: doc_03
Risk flags: []
Hedging detected: False
Sentiment: neutral
```

```
=====
Document: doc_04
Risk flags: []
Hedging detected: True
Sentiment: cautious
```

```
=====
Document: doc_05
Risk flags: []
Hedging detected: False
Sentiment: confident
```

```
=====
Document: doc_06
Risk flags: ['regulatory']
Hedging detected: False
Sentiment: neutral
```

```
# =====
# PART C – MULTI-AGENT DEBATE DEMO
# =====

def bull_agent(ticker):
    data = STOCK_UNIVERSE[ticker]

    return (
        f"Bull view: {ticker} has an analyst expected return of "
        f"{data['analyst_expected_return']:.1%} and a beta of "
        f"{data['beta']:.2f}, suggesting attractive return potential "
        f"relative to market sensitivity."
    )

def bear_agent(ticker):
    data = STOCK_UNIVERSE[ticker]

    return (
        f"Bear view: {ticker} has a standard deviation of "
        f"{data['std_dev']:.1%}, indicating meaningful volatility "
        f"and therefore higher investment risk."
    )

def synthesizer_agent(ticker, bull_argument, bear_argument):
    data = STOCK_UNIVERSE[ticker]

    return (
        f"Balanced view: {ticker} offers an analyst expected return of "
        f"{data['analyst_expected_return']:.1%} with a beta of "
        f"{data['beta']:.2f}, but its standard deviation of "
```

```

        f"{data['std_dev']:.1%} highlights the associated risk. "
        f"Investors should balance the return opportunity against "
        f"the stock's volatility."
    )

def run_debate(ticker):
    """
    Run the 3-agent debate in mock mode.
    """

    bull = bull_agent(ticker)
    bear = bear_agent(ticker)

    synthesis = synthesizer_agent(
        ticker,
        bull,
        bear
    )

    return {
        "ticker": ticker,
        "bull": bull,
        "bear": bear,
        "synthesizer": synthesis
    }

# =====
# RUN DEBATE
# =====

# Choose one ticker from STOCK_UNIVERSE
debate_ticker = "PAYFIN"

debate_result = run_debate(debate_ticker)

print("=" * 70)
print("MULTI-AGENT DEBATE")
print("=" * 70)

print("\nTicker:", debate_result["ticker"])

print("\n🐮 BULL AGENT")
print(debate_result["bull"])

print("\n🐻 BEAR AGENT")
print(debate_result["bear"])

print("\n🧠 SYNTHESIZER")
print(debate_result["synthesizer"])

=====
MULTI-AGENT DEBATE
=====

Ticker: PAYFIN

🐮 BULL AGENT
Bull view: PAYFIN has an analyst expected return of 16.0% and a beta of 1.35, suggesting att

🐻 BEAR AGENT
Bear view: PAYFIN has a standard deviation of 28.0%, indicating meaningful volatility and th

```

 SYNTHESIZER

Balanced view: PAYFIN offers an analyst expected return of 16.0% with a beta of 1.35, but it

```
# =====
# PART D – DCF VALUATION CALCULATOR
# =====

import pandas as pd
import numpy as np

# -----
# 1. DCF INPUTS
# -----

# Illustrative hypothetical Paytm business line
base_fcff = 100_000_000      # INR
growth_rates = [0.12, 0.10, 0.08, 0.07, 0.06]

tax_rate = 0.25
beta = STOCK_UNIVERSE["PAYFIN"]["beta"]

risk_free_rate = RISK_FREE_RATE
market_return = MARKET_RETURN

# Cost of equity using CAPM
cost_of_equity = (
    risk_free_rate
    + beta * (market_return - risk_free_rate)
)

# Illustrative after-tax cost of debt
pre_tax_cost_of_debt = 0.09
after_tax_cost_of_debt = (
    pre_tax_cost_of_debt * (1 - tax_rate)
)

# Capital structure
equity_weight = 0.70
debt_weight = 0.30

# WACC
wacc = (
    equity_weight * cost_of_equity
    + debt_weight * after_tax_cost_of_debt
)

terminal_growth = 0.03

# -----
# 2. FCFF PROJECTION
# -----

fcff_values = []

current_fcff = base_fcff

for growth in growth_rates:
    current_fcff = current_fcff * (1 + growth)
    fcff_values.append(current_fcff)
```

```

projection_years = [1, 2, 3, 4, 5]

fcff_df = pd.DataFrame({
    "Year": projection_years,
    "Growth Rate": growth_rates,
    "FCFF (INR)": fcff_values
})

# -----
# 3. TERMINAL VALUE
# -----

terminal_fcff = fcff_values[-1] * (1 + terminal_growth)

terminal_value = (
    terminal_fcff
    / (wacc - terminal_growth)
)

# -----
# 4. DISCOUNT FCFF AND TERMINAL VALUE
# -----

discount_factors = [
    1 / ((1 + wacc) ** year)
    for year in projection_years
]

pv_fcff = [
    fcff * discount_factor
    for fcff, discount_factor
    in zip(fcff_values, discount_factors)
]

pv_terminal_value = (
    terminal_value
    * discount_factors[-1]
)

enterprise_value = (
    sum(pv_fcff)
    + pv_terminal_value
)

# -----
# 5. EV / EBITDA CROSS-CHECK
# -----

illustrative_ebitda = 150_000_000
illustrative_multiple = 12

ev_ebitda_value = (
    illustrative_ebitda
    * illustrative_multiple
)

# -----
# 6. PRINT DCF RESULTS

```

```
# -----

print("=" * 70)
print("DCF VALUATION")
print("=" * 70)

print(f"\nRisk-free rate: {risk_free_rate:.2%}")
print(f"Market return: {market_return:.2%}")
print(f"Beta used: {beta:.2f}")
print(f"Cost of equity: {cost_of_equity:.2%}")
print(f"After-tax cost of debt: {after_tax_cost_of_debt:.2%}")
print(f"WACC: {wacc:.2%}")
print(f"Terminal growth: {terminal_growth:.2%}")

print("\nFCFF PROJECTION")
display(fcff_df)

print(f"\nTerminal FCFF: INR {terminal_fcff:,.0f}")
print(f"Terminal Value: INR {terminal_value:,.0f}")
print(f"PV of Terminal Value: INR {pv_terminal_value:,.0f}")
print(f"PV of projected FCFF: INR {sum(pv_fcff):,.0f}")

print(f"\nDCF Enterprise Value: INR {enterprise_value:,.0f}")

print("\nEV / EBITDA CROSS-CHECK")
print(f"Illustrative EBITDA: INR {illustrative_ebitda:,.0f}")
print(f"Illustrative EV/EBITDA multiple: {illustrative_multiple:.1f}x")
print(f"EV/EBITDA Enterprise Value: INR {ev_ebitda_value:,.0f}")

print("\nDCF vs EV/EBITDA difference:")
print(
    f"INR {enterprise_value - ev_ebitda_value:,.0f}"
)
```

DCF VALUATION

Risk-free rate: 7.00%
 Market return: 13.00%
 Beta used: 1.35
 Cost of equity: 15.10%
 After-tax cost of debt: 6.75%
 WACC: 12.60%
 Terminal growth: 3.00%

FCFF PROJECTION

	Year	Growth Rate	FCFF (INR)	
0	1	0.12	112000000.0	
1	2	0.10	123200000.0	
2	3	0.08	133056000.0	
3	4	0.07	142369920.0	
4	5	0.06	150912115.2	

Terminal FCFF: INR 155,439,479
 Terminal Value: INR 1,620,004,989
 PV of Terminal Value: INR 895,201,550
 PV of projected FCFF: INR 461,837,773

DCF Enterprise Value: INR 1,357,039,324

EV / EBITDA CROSS-CHECK

Illustrative EBITDA: INR 150,000,000
 Illustrative EV/EBITDA multiple: 12.0x
 EV/EBITDA Enterprise Value: INR 1,800,000,000

DCF vs EV/EBITDA difference:
 INR -442,960,676

Next steps:

[Generate code with fcff_df](#)

[New interactive sheet](#)

```

# =====
# DCF SENSITIVITY ANALYSIS - 3 x 3 GRID
# =====

wacc_values = [
    wacc - 0.01,
    wacc,
    wacc + 0.01
]

growth_values = [
    terminal_growth - 0.01,
    terminal_growth,
    terminal_growth + 0.01
]

sensitivity = pd.DataFrame(
    index=[f"WACC {x:.2%}" for x in wacc_values],
    columns=[f"Growth {x:.2%}" for x in growth_values]
)
  
```

```

for w in wacc_values:

    for g in growth_values:

        # Terminal value under this scenario
        scenario_terminal_fcff = (
            fcff_values[-1] * (1 + g)
        )

        scenario_terminal_value = (
            scenario_terminal_fcff
            / (w - g)
        )

        # Discount projected FCFF
        scenario_pv_fcff = sum(
            fcff_values[year - 1]
            / ((1 + w) ** year)
            for year in projection_years
        )

        # Discount terminal value
        scenario_pv_terminal = (
            scenario_terminal_value
            / ((1 + w) ** 5)
        )

        scenario_ev = (
            scenario_pv_fcff
            + scenario_pv_terminal
        )

        sensitivity.loc[
            f"WACC {w:.2%}",
            f"Growth {g:.2%}"
        ] = scenario_ev

# Convert values to numbers
sensitivity = sensitivity.astype(float)

print("=" * 70)
print("DCF 3 x 3 SENSITIVITY TABLE")
print("=" * 70)

display(
    sensitivity.round(0)
)

# -----
# REQUIRED SELF-CHECK
# -----

min_wacc_minus_growth = min(
    w - g
    for w in wacc_values
    for g in growth_values
)

print(
    f"\nMinimum WACC - terminal growth spread: "

```

```

    f"{min_wacc_minus_growth:.2%}"
)

if min_wacc_minus_growth >= 0.01:
    print(
        "SELF-CHECK PASSED: "
        "WACC exceeds terminal growth by at least "
        "1 percentage point in every sensitivity cell."
    )
else:
    print(
        "SELF-CHECK FAILED: "
        "WACC does not exceed terminal growth sufficiently."
    )

```

```

=====
DCF 3 x 3 SENSITIVITY TABLE
=====

```

	Growth 2.00%	Growth 3.00%	Growth 4.00%	
WACC 11.60%	1.400977e+09	1.518970e+09	1.668034e+09	
WACC 12.60%	1.264676e+09	1.357039e+09	1.470896e+09	
WACC 13.60%	1.152005e+09	1.225781e+09	1.314935e+09	

Minimum WACC - terminal growth spread: 7.60%

SELF-CHECK PASSED: WACC exceeds terminal growth by at least 1 percentage point in every sens

```

# =====
# EV / EBITDA CROSS-CHECK
# =====

# Illustrative assumptions for the cross-check
ebitda = 100_000_000
ev_ebitda_multiple = 12

# Enterprise value using EV/EBITDA
ev_ebitda_value = ebitda * ev_ebitda_multiple

# DCF Enterprise Value
# Based on the DCF values calculated above:
# PV of projected FCFF = INR 461,837,773
# PV of terminal value = INR 895,201,550

pv_projected_fcff_crosscheck = 461_837_773
pv_terminal_value_crosscheck = 895_201_550

dcf_enterprise_value = (
    pv_projected_fcff_crosscheck
    + pv_terminal_value_crosscheck
)

# Difference between the two approaches
difference = ev_ebitda_value - dcf_enterprise_value

difference_pct = (
    difference / dcf_enterprise_value
) * 100

print("=" * 70)
print("EV / EBITDA CROSS-CHECK")

```



```
print("=" * 70)

print(f"Illustrative EBITDA: INR {ebitda:,.0f}")
print(f"EV/EBITDA multiple: {ev_ebitda_multiple:.1f}x")

print(f"\nDCF Enterprise Value: INR {dcf_enterprise_value:,.0f}")

print(
    f"EV/EBITDA Enterprise Value: "
    f"INR {ev_ebitda_value:,.0f}"
)

print(f"\nDifference: INR {difference:,.0f}")

print(
    f"Difference (% of DCF value): "
    f"{difference_pct:.2f}%"
)

print("\nInterpretation:")
print(
    "The DCF valuation estimates enterprise value using "
    "projected free cash flows, WACC and terminal growth."
)

print(
    "The EV/EBITDA approach provides a market-multiple "
    "cross-check, so the difference reflects the assumptions "
    "and valuation methodology used "
```